

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«Алгоритми та структури даних»

ЗВІТ
з практичної роботи №5

Виконав:
Студент групи КН-25-1
Гнатюк Є.М.
Перевірила:
Доцент кафедри КІЕ
Сидоренко В.М.

Кременчук 2026

Тема: **Графи. Ациклічні графи**

Мета: набути практичних навичок розв'язання задач топографічного сортування та оцінювання їх асимптотичної складності.

4. Задано ациклічний граф: $\{2, 4, 7, 10, 12\}, \{(2, 7), (4, 10), (7, 10), (4, 12)\}$. Побудувати граф і розв'язати задачу топологічного сортування за допомогою алгоритму DFS.

```
vertices = [2, 4, 7, 10, 12]
```

```
edges = [(2, 7), (4, 10), (7, 10), (4, 12)]
```

```
G = nx.DiGraph()
```

```
G.add_nodes_from(vertices)
```

```
G.add_edges_from(edges)
```

```
def topological_sort_dfs():
```

```
    visited = set()
```

```
    stack = []
```

```
    def dfs(node):
```

```
        visited.add(node)
```

```
        for neighbor in G.neighbors(node):
```

```
            if neighbor not in visited:
```

```
                dfs(neighbor) stack.append(node)
```

```
    for vertex in G.nodes():
```

```
        if vertex not in visited:
```

```
            dfs(vertex) return stack[::-1]
```

1. Беремо першу вільну вершину, з якої нічого не виходить або яка просто стоїть першою в черзі — наприклад, вершину 2.

2. Дивимося, куди вона веде. Вона веде до 7. Йдемо туди.

3. Перевіряємо, куди веде 7. Вона веде до 10. Йдемо далі.

4. Дивимось на 10 — з неї нікуди йти. Це «тупик». Ми завершили з цією гілкою. Записуємо 10 в кінець списку.
5. Повертаємось до 7. Ми вже перевірили всі її шляхи (тільки до 10). Вона теж завершена. Додаємо 7 перед 10.
6. Повертаємось до 2. Всі її шляхи (до 7) оброблені. Додаємо 2 перед 7.
7. Шукаємо наступну невідвідану вершину — це вершина 4.
8. Занурення (вершина 4): Вона веде до 10 та 12.
 - Дивимось на 10 — вона вже «готова» і відвідана, ігноруємо.
 - Йдемо до 12.
9. З 12 нікуди йти. Завершуємо її. Додаємо 12 у наш список перед уже обробленими вершинами за логікою занурення.
10. Всі шляхи від 4 (до 10 та 12) перевірені. Вона готова. Додаємо 4 на самий початок.

Контрольні запитання:

1. Які переваги і недоліки алгоритму Кана порівняно з алгоритмом DFS для топологічного сортування графа?

- *Переваги:* Алгоритм Кана працює ітеративно (без ризику переповнення стеку рекурсії) та дозволяє дуже легко і в процесі роботи виявити наявність циклів у графі.
- *Недоліки:* Потребує додаткової пам'яті та часу на попередній підрахунок кількості вхідних ребер для кожної вершини.

2. Яка складність часу і пам'яті для кожного з алгоритмів у найгіршому і найкращому випадках? Для обох алгоритмів (Кана та DFS) часова складність у всіх випадках є лінійною і становить $O(V + E)$, де V — вершини, а E — ребра графа. Просторова складність (пам'ять) для обох методів становить $O(V)$.

3. Чи можна застосовувати алгоритм Кана до графів з вагами на ребрах? Як це порівняти з DFS? Так, обидва алгоритми можна застосовувати до зважених графів. Проте ваги ребер під час топологічного сортування повністю ігноруються, оскільки обидва методи працюють виключно зі структурою залежностей (напрямами стрілок).

4. Як впливає структура графа на швидкість роботи кожного з цих алгоритмів? Розріджені графи (де мало ребер) обробляються швидше в обох

випадках. Щільні граfi збільшують кількість елементарних операцій через велику кількість зв'язків, але загальна теоретична складність в обох випадках залишається незмінною — $O(V + E)$.

5. Чи є обмеження використання кожного алгоритму для певних типів графів або завдань? Обидва алгоритми мають суворе обмеження: вони працюють лише з орієнтованими ациклічними графами (DAG). Наявність хоча б одного замкненого циклу в графі робить топологічне сортування математично неможливим.

6. Які варіанти оптимізації можна застосувати для кожного алгоритму з метою поліпшення його продуктивності?

- *Для Кана:* Використання ефективної структури черги (deque) для швидкого вилучення вершин та масиву для збереження степенів.
- *Для DFS:* Використання ітеративної версії алгоритму на основі явного стеку замість системної рекурсії, щоб уникнути перевантаження викликів.